Programação de Sistemas Computacionais

Tema: Sistema de Gestão de um Posto de Saúde

1. CONTEXTUALIZAÇÃO

Um posto de saúde necessita de um sistema informatizado para organizar os dados dos pacientes, dos profissionais de saúde e dos agendamentos de consultas. Na primeira etapa da avaliação, foi realizada a modelagem do banco de dados e a criação das estruturas necessárias para armazenar essas informações.

Nesta segunda etapa, você deverá desenvolver a aplicação que permitirá aos usuários interagir com o banco de dados, implementando as funcionalidades de autenticação, navegação e gerenciamento das informações.

A aplicação deverá utilizar o banco de dados desenvolvido na primeira etapa, respeitando as tabelas, os relacionamentos e as restrições definidos no projeto. Também deverão ser considerados os cuidados com a segurança da informação, a proteção dos dados pessoais e a validação das operações realizadas pelo usuário.

2. DESAFIO

Você deverá desenvolver uma aplicação para o posto de saúde, utilizando a linguagem de programação e o SGBD definidos na documentação técnica da primeira etapa.

A aplicação deverá permitir o acesso autenticado ao sistema e disponibilizar funcionalidades para consultar, cadastrar e atualizar os registros armazenados no banco de dados.

3. REQUISITOS DA APLICAÇÃO

3.1. Funcionalidade de login

- Implementar uma tela de autenticação com campos para usuário e senha.
- Validar as credenciais informadas utilizando os dados registrados no banco de dados.
- Permitir o acesso ao sistema somente quando a autenticação for válida.
- Exibir uma mensagem adequada quando houver falha na autenticação.
- Criar uma sessão para o usuário autenticado, estabelecendo um tempo de expiração.
- Armazenar as senhas de forma segura, utilizando mecanismo apropriado de hash de senha.

3.2. Funcionalidade principal do sistema

Após a autenticação, o sistema deverá apresentar uma página principal contendo:

- O nome do usuário autenticado.
- Uma opção para acessar o gerenciamento de pacientes.
- Uma opção para acessar o gerenciamento de profissionais de saúde.
- Uma opção para acessar o gerenciamento de agendamentos.
- Uma opção para encerrar a sessão e sair do sistema.

3.3. Gerenciamento de pacientes

A funcionalidade deverá permitir:

- Listar os pacientes cadastrados no banco de dados.
- Realizar a busca de pacientes utilizando pelo menos um campo, como nome ou CPF.
- Cadastrar um novo paciente, salvando os dados no banco de dados.
- Editar os dados de um paciente já cadastrado, persistindo as alterações.
- Excluir um paciente, respeitando as restrições de integridade referencial do banco de dados.

3.4. Gerenciamento de profissionais de saúde

A funcionalidade deverá permitir:

- Listar os profissionais de saúde cadastrados.
- Consultar as informações de um profissional.
- Cadastrar um novo profissional, persistindo os dados no banco de dados.
- Atualizar os dados de um profissional já cadastrado.

3.5. Gerenciamento de agendamentos

A funcionalidade deverá permitir:

- Listar os agendamentos cadastrados, ordenados por data e horário.
- Exibir os dados do paciente e do profissional relacionados a cada consulta.
- Atualizar a data e o horário de um agendamento, salvando as alterações no banco de dados.
- Preservar a integridade dos relacionamentos entre agendamentos, pacientes e profissionais.

4. REGRAS DE SEGURANÇA E QUALIDADE

- Validar os dados informados nos formulários antes de realizar operações no banco de dados.
- Utilizar consultas parametrizadas ou mecanismos equivalentes para evitar SQL Injection.
- Não armazenar senhas em texto puro.
- Não exibir desnecessariamente o CPF e outros dados pessoais nas listagens.
- Restringir o acesso às funcionalidades internas a usuários autenticados.
- Encerrar corretamente a sessão ao sair do sistema.
- Tratar erros de operação sem expor informações técnicas sensíveis ao usuário.

5. RESULTADOS E ENTREGAS ESPERADAS

Crie um diretório compactado na área de trabalho, no formato `.zip`, `.rar` ou `.7zip`, nomeado com seu nome completo, contendo os seguintes arquivos:

Nº	Entrega	Tipo de entrega
1	Código-fonte da funcionalidade de login	Diretório sistema
2	Código-fonte da funcionalidade principal	Diretório sistema
3	Código-fonte do gerenciamento de pacientes	Diretório sistema
4	Código-fonte do gerenciamento de profissionais de saúde	Diretório sistema
5	Código-fonte do gerenciamento de agendamentos	Diretório sistema
6	Arquivos necessários para executar a aplicação e instruções de execução	Diretório sistema

O diretório deverá conter os arquivos necessários para executar a aplicação, sem incluir senhas reais, credenciais pessoais ou outros segredos de configuração.

6. CRITÉRIOS GERAIS DE AVALIAÇÃO

Serão considerados:

- Funcionamento da autenticação e tratamento de falhas.
- Controle de sessão e acesso às funcionalidades.
- Integração com o banco de dados criado na primeira etapa.
- Funcionamento das operações de consulta, cadastro e atualização.
- Implementação das buscas e do ordenamento dos agendamentos.
- Respeito à integridade referencial e às regras de negócio.
- Aplicação de práticas básicas de segurança e validação.
- Organização e entrega dos arquivos do projeto.

ANEXO - GUIA DO ESTUDANTE

Programação back-end com PHP e MySQL

Cenário: Sistema de Gestão de um Posto de Saúde

1. Objetivo da atividade

Nesta etapa, você deverá implementar, em PHP, funcionalidades que utilizem o banco de dados desenvolvido na Parte 1. O foco da avaliação é o processamento no back-end, a persistência dos dados, a autenticação, o controle de sessão e a aplicação de práticas básicas de segurança. Não será avaliada a criação de um front-end elaborado; formulários HTML simples são suficientes quando necessários para enviar ou apresentar dados.

Importante: utilize o banco de dados e os nomes de tabelas e colunas definidos no seu projeto da Parte 1. Os exemplos deste guia são modelos de referência: adapte-os ao seu esquema real.

2. O que deve ser desenvolvido

2.1 Autenticação e sessão

- Receber usuário/login e senha enviados por formulário.
- Consultar o usuário no banco de dados por meio de consulta parametrizada.
- Verificar a senha com password_verify(); não comparar a senha em texto puro.
- Em caso de sucesso, iniciar a sessão e guardar nela apenas os dados necessários, como o identificador e o nome do usuário.
- Em caso de falha, apresentar uma mensagem genérica, sem revelar se o login ou a senha está incorreto.
- Proteger as páginas internas para que somente usuários autenticados possam acessá-las.
- Implementar logout, limpar os dados da sessão e encerrar a sessão.

2.2 Página principal

- Exibir o nome do usuário autenticado.
- Disponibilizar acesso às funcionalidades de pacientes, profissionais de saúde e agendamentos.
- Disponibilizar uma opção para sair do sistema.

2.3 Gerenciamento de pacientes

- Listar pacientes cadastrados.
- Buscar pacientes por pelo menos um campo, como nome ou CPF.
- Cadastrar um paciente e persistir os dados no banco.
- Editar um paciente existente e salvar as alterações.
- Caso a exclusão seja exigida no enunciado, respeitar as restrições de integridade referencial. Não apagar registros relacionados de forma automática sem regra definida.

2.4 Gerenciamento de profissionais de saúde

- Listar profissionais cadastrados.
- Cadastrar um profissional e persistir os dados.
- Consultar e atualizar os dados de um profissional existente.

2.5 Gerenciamento de agendamentos

- Listar os agendamentos ordenados por data e horário.
- Exibir, quando aplicável, o nome do paciente e do profissional associado a cada agendamento.
- Atualizar a data e o horário de um agendamento existente.
- Manter válidos os relacionamentos com pacientes e profissionais.

3. Estrutura sugerida de pastas

Você pode usar uma estrutura semelhante à abaixo. Os arquivos podem ser organizados de outra forma, desde que o projeto permaneça claro e funcional.

```
posto_saude/
├── config/
│   └── conexao.php
├── includes/
│   └── funcoes.php
├── auth/
│   ├── login.php
│   └── logout.php
├── pacientes/
│   ├── listar.php
│   ├── cadastrar.php
│   ├── buscar.php
│   └── editar.php
├── profissionais/
│   ├── listar.php
│   ├── cadastrar.php
│   └── editar.php
├── agendamentos/
│   ├── listar.php
│   └── atualizar.php
├── index.php
└── README.md
```

A estrutura é uma sugestão, não uma exigência de nomes. O projeto deve conter todos os arquivos necessários para executar as funcionalidades solicitadas.

4. Preparação do ambiente

1. Confirme que o servidor web, o PHP e o MySQL/MariaDB estão instalados e funcionando no laboratório.
2. Copie o projeto para o diretório servido pelo ambiente local (por exemplo, htdocs no XAMPP).
3. Importe ou selecione o banco criado na Parte 1.
4. Confira o nome do banco, as tabelas, as colunas e as chaves estrangeiras antes de escrever as consultas.
5. Configure as credenciais locais em config/conexao.php. Use somente credenciais de laboratório fornecidas para a atividade.
6. Teste primeiro a conexão; depois implemente e teste uma funcionalidade por vez.

5. Conexão com o banco usando PDO

O arquivo de conexão pode centralizar a criação do objeto PDO. Ajuste banco, usuário e senha ao ambiente fornecido.

```php
<?php
$host = 'localhost';
$banco = 'posto_saude_db'; // ajuste ao nome real
$usuario = 'root';         // ajuste ao ambiente
$senha = '';               // ajuste ao ambiente

try {
    $pdo = new PDO(
        "mysql:host=$host;dbname=$banco;charset=utf8mb4",
        $usuario,
        $senha,
        [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES => false
        ]
    );
} catch (PDOException $e) {
    error_log($e->getMessage());
    exit('Não foi possível conectar ao banco de dados.');
}
```

Em um ambiente real, não exiba detalhes técnicos da exceção ao usuário. O error_log() registra o detalhe para diagnóstico sem mostrá-lo na página.

6. Consultas parametrizadas

Use prepare() e execute() para enviar valores à consulta. Não monte SQL concatenando diretamente dados recebidos do usuário.

```
<?php
require_once __DIR__ . '/../config/conexao.php';

// Exemplo: buscar por nome.
// Ajuste tabela e coluna ao seu banco.
$nome = trim($_GET['nome'] ?? '');

$sql = 'SELECT id_paciente, nome
        FROM pacientes
        WHERE nome LIKE :nome
        ORDER BY nome';

$stmt = $pdo->prepare($sql);
$stmt->execute(['nome' => '%' . $nome . '%']);
$pacientes = $stmt->fetchAll();
```

Os nomes id_paciente, pacientes e nome são ilustrativos. Substitua-os pelos nomes reais do seu esquema. Parâmetros protegem valores; nomes de tabelas e colunas não devem ser recebidos livremente do usuário.

7. Padrões para operações no banco

7.1 Inserir um registro

```
$sql = 'INSERT INTO pacientes (nome, telefone)
        VALUES (:nome, :telefone)';

$stmt = $pdo->prepare($sql);
$stmt->execute([
    'nome' => $nome,
    'telefone' => $telefone
]);
```

Antes do INSERT, valide os campos obrigatórios e os formatos esperados. Os campos do exemplo devem existir na tabela.

7.2 Atualizar um registro

```
$sql = 'UPDATE pacientes
        SET nome = :nome, telefone = :telefone
        WHERE id_paciente = :id';

$stmt = $pdo->prepare($sql);
$stmt->execute([
    'nome' => $nome,
    'telefone' => $telefone,
    'id' => $idPaciente
]);
```

A condição WHERE é essencial para evitar que a operação altere todos os registros.

7.3 Listar com ordenação

```
$sql = 'SELECT id_agendamento, data_consulta, horario
        FROM agendamentos
        ORDER BY data_consulta ASC, horario ASC';

$stmt = $pdo->query($sql);
$agendamentos = $stmt->fetchAll();
```

Adapte tabela e colunas. Para exibir paciente e profissional na mesma listagem, utilize JOIN conforme os relacionamentos do seu modelo.

7.4 Relacionar tabelas com JOIN

```
SELECT a.data_consulta, a.horario,
       p.nome AS paciente,
       pr.nome AS profissional
FROM agendamentos AS a
JOIN pacientes AS p
  ON p.id_paciente = a.id_paciente
JOIN profissionais AS pr
  ON pr.id_profissional = a.id_profissional
ORDER BY a.data_consulta, a.horario;
```

Este exemplo pressupõe chaves estrangeiras com esses nomes. Confira o DER e o script SQL antes de utilizá-lo.

8. Senhas: gerar e verificar hash

Senhas não devem ser armazenadas em texto puro nem protegidas com criptografia reversível. Use as funções específicas de hash de senha do PHP.

```
// Ao cadastrar ou redefinir uma senha:
$hash = password_hash($senhaInformada, PASSWORD_DEFAULT);

// Ao autenticar:
if (password_verify($senhaInformada, $usuario['senha_hash'])) {
    // Senha válida
} else {
    // Credenciais inválidas
}
```

O nome senha_hash é ilustrativo. A coluna deve comportar o hash gerado (VARCHAR(255) é uma escolha comum). A verificação da senha ocorre no PHP; não é necessário descriptografar o hash.

Pré-requisito: o banco precisa possuir uma tabela de usuários e um registro de teste com a senha armazenada como hash. Se isso não estiver previsto no banco da Parte 1, confirme com o professor como será disponibilizado o usuário de teste antes de implementar o login.

9. Sessão e proteção de páginas

```
<?php
// No início do arquivo, antes de qualquer saída:
session_start();

// Após autenticação bem-sucedida:
session_regenerate_id(true);
$_SESSION['id_usuario'] = $usuario['id_usuario'];
$_SESSION['nome_usuario'] = $usuario['nome'];

// Em cada página protegida:
if (!isset($_SESSION['id_usuario'])) {
    header('Location: /posto_saude/auth/login.php');
    exit;
}
```

Ajuste os caminhos ao seu projeto. session_start() deve ocorrer antes de enviar HTML ou espaços ao navegador. Guarde na sessão apenas os dados necessários, não a senha nem o hash da senha.

9.1 Logout

```
<?php
session_start();
$_SESSION = [];
session_destroy();

header('Location: login.php');
exit;
```

Ajuste o caminho de redirecionamento. O logout deve ser acionado por uma requisição apropriada e não apenas ocultar um link.

10. Validação e segurança

- Validar no back-end os campos obrigatórios e os formatos, mesmo que haja validação no formulário.
- Usar consultas preparadas para valores recebidos do usuário.
- Escapar texto ao exibi-lo em HTML, por exemplo com htmlspecialchars($valor, ENT_QUOTES, 'UTF-8').
- Não exibir mensagens de erro do banco, caminhos internos, senhas ou credenciais ao usuário.
- Não armazenar senhas em texto puro.
- Não expor CPF em listagens quando ele não for necessário para a finalidade.
- Não permitir acesso às páginas internas sem sessão autenticada.
- Usar POST para operações que alteram dados; não executar cadastro, edição ou exclusão por links GET.
- Respeitar chaves estrangeiras e restrições do banco.
- Não colocar credenciais reais no README, no código enviado ou em repositórios públicos.

11. Como testar o trabalho

- A conexão funciona e o projeto acessa o banco correto.
- O login aceita credenciais válidas e rejeita credenciais inválidas.
- Uma página interna não pode ser acessada sem autenticação.
- O logout encerra a sessão.
- A listagem apresenta os registros existentes.
- A busca retorna resultados coerentes e trata uma busca sem resultados.
- O cadastro persiste um novo registro e valida os campos obrigatórios.
- A edição altera somente o registro selecionado.
- Os agendamentos aparecem em ordem de data e horário.
- A atualização de um agendamento persiste no banco.
- As consultas usam parâmetros e a saída textual é tratada com segurança.
- O sistema apresenta mensagens compreensíveis quando ocorre uma falha.

12. Entrega

Compacte o projeto em um arquivo ZIP, RAR ou 7Z com seu nome completo. Inclua os arquivos PHP necessários, o README.md com instruções de execução e, se solicitado pelo professor, os arquivos de configuração de exemplo. Não inclua senhas reais, arquivos temporários ou cópias desnecessárias do banco.

O README deve informar:

- Versões ou ferramentas utilizadas no laboratório, quando conhecidas.
- Nome do banco de dados esperado e como importá-lo.
- Como configurar a conexão local.
- Como iniciar e acessar a aplicação.
- Usuário de teste fornecido para a avaliação, sem registrar uma senha pessoal ou real.

13. Checklist final do estudante

☐ Meu projeto usa o banco desenvolvido na Parte 1.
☐ Adaptei tabelas e colunas dos exemplos ao meu modelo.
☐ A conexão PDO funciona.
☐ Implementei as funcionalidades exigidas no enunciado.
☐ Testei os cenários de sucesso e de falha.
☐ Usei consultas parametrizadas e hash de senha.
☐ Protegi as páginas internas com sessão.
☐ O projeto está organizado e contém instruções de execução.
☐ Compactei os arquivos com meu nome completo.

Nota sobre os exemplos

Os trechos deste guia são exemplos de referência, não um sistema completo. Os nomes das tabelas, colunas, chaves e caminhos devem ser conferidos e adaptados ao banco e ao ambiente da avaliação. O estudante continua responsável por implementar, integrar, testar e explicar o próprio código.